

GenAI Code Comparison & Analysis: C++ CEO Email Priority Queue

Author: Kaleb Blakemore

KUID: 3228599

Course: EECS 348 Software Engineering I

Date: September 17, 2026

GenAI Programs Used

OpenAI ChatGPT, accessed through the web browser chat interface.

Google Gemini, accessed through the web browser chat interface.

How I Accessed Them

I used both ChatGPT and Google Gemini through their web-based chat interfaces on my Mac. I started a new conversation with each one so that neither program had previous information about my assignment or previous code. This gave both programs the same starting point when generating their code.

Shared Prompt Used

I gave both GenAI programs the exact same prompt so that I could compare the code they produced fairly.

Prompt:

"Write a C program that implements a custom MaxHeap-based priority queue from scratch (without using pre-existing heap libraries). The program must manage a CEO's incoming emails based on sender category hierarchy and recency.

- The program will prioritize emails for a busy company CEO.
- Use a MaxHeap as a list-based implementation for the priority queue.
- Create all functions from scratch without pre-existing heap modules.
- The sender categories and priority order are: Boss (highest), Subordinate, Peer, ImportantPerson, and OtherPerson (lowest).
- If categories match, newer dates (format MM-DD-YYYY) must be prioritized over older dates.
- It should parse standard commands from standard input: EMAIL, NEXT, READ, and COUNT."

Generated Code

ChatGPT Generated Code: Baseline

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000

typedef struct {
    char sender[100];
    char category[30];
    char date[11];
    char subject[200];
} Email;

typedef struct {
    Email emails[MAX_EMAILS];
    int size;
} MaxHeap;

/* Get the priority value for each sender category */
int get_category_priority(char category[]) {
    if (strcmp(category, "Boss") == 0)
        return 5;
    else if (strcmp(category, "Subordinate") == 0)
        return 4;
    else if (strcmp(category, "Peer") == 0)
        return 3;
    else if (strcmp(category, "ImportantPerson") == 0)
        return 2;
    else
        return 1;
}

/* Convert MM-DD-YYYY into YYYYMMDD so dates can be compared */
int get_date_value(char date[]) {
    int month, day, year;

    sscanf(date, "%d-%d-%d", &month, &day, &year);

    return year * 10000 + month * 100 + day;
}

/* Returns 1 if email1 has higher priority than email2 */
int higher_priority(Email email1, Email email2) {
    int priority1 = get_category_priority(email1.category);
    int priority2 = get_category_priority(email2.category);

    if (priority1 > priority2)
        return 1;
}

```

```

    if (priority1 < priority2)
        return 0;

    /* If categories match, compare dates */
    if (get_date_value(email1.date) > get_date_value(email2.date))
        return 1;

    return 0;
}

/* Swap two emails */
void swap_emails(Email *email1, Email *email2) {
    Email temp = *email1;
    *email1 = *email2;
    *email2 = temp;
}

/* Move an email up the heap */
void heapify_up(MaxHeap *heap, int index) {
    int parent;

    while (index > 0) {
        parent = (index - 1) / 2;

        if (higher_priority(heap->emails[index],
                           heap->emails[parent])) {
            swap_emails(&heap->emails[index],
                       &heap->emails[parent]);

            index = parent;
        } else {
            break;
        }
    }
}

/* Move an email down the heap */
void heapify_down(MaxHeap *heap, int index) {
    int left;
    int right;
    int largest;

    while (1) {
        left = 2 * index + 1;
        right = 2 * index + 2;
        largest = index;

        if (left < heap->size &&

```

```

        higher_priority(heap->emails[left],
                        heap->emails[largest])) {
            largest = left;
        }

        if (right < heap->size &&
            higher_priority(heap->emails[right],
                            heap->emails[largest])) {
            largest = right;
        }

        if (largest != index) {
            swap_emails(&heap->emails[index],
                       &heap->emails[largest]);

            index = largest;
        } else {
            break;
        }
    }
}

/* Add an email to the MaxHeap */
void insert_email(MaxHeap *heap, Email email) {
    if (heap->size >= MAX_EMAILS) {
        printf("Inbox is full.\n");
        return;
    }

    heap->emails[heap->size] = email;
    heap->size++;

    heapify_up(heap, heap->size - 1);
}

/* Remove and return the highest priority email */
Email remove_max(MaxHeap *heap) {
    Email top = heap->emails[0];

    heap->emails[0] = heap->emails[heap->size - 1];
    heap->size--;

    if (heap->size > 0)
        heapify_down(heap, 0);

    return top;
}

```

```

/* Display the next highest priority email */
void next_email(MaxHeap *heap) {
    if (heap->size == 0) {
        printf("No emails in inbox.\n");
        return;
    }

    printf("%s %s %s %s\n",
        heap->emails[0].sender,
        heap->emails[0].category,
        heap->emails[0].date,
        heap->emails[0].subject);
}

/* Read and remove the highest priority email */
void read_email(MaxHeap *heap) {
    Email email;

    if (heap->size == 0) {
        printf("No emails in inbox.\n");
        return;
    }

    email = remove_max(heap);

    printf("%s %s %s %s\n",
        email.sender,
        email.category,
        email.date,
        email.subject);
}

/* Print the number of emails currently in the inbox */
void count_emails(MaxHeap *heap) {
    printf("%d\n", heap->size);
}

int main(void) {
    MaxHeap heap;
    char command[20];
    Email email;

    heap.size = 0;

    while (scanf("%s", command) != EOF) {

        if (strcmp(command, "EMAIL") == 0) {

```

```

        /*
        * Expected format:
        * EMAIL sender category date subject
        */
        scanf("%s", email.sender);
        scanf("%s", email.category);
        scanf("%s", email.date);

        /*
        * Read the rest of the line as the subject.
        */
        getchar();
        fgets(email.subject, sizeof(email.subject), stdin);

        /* Remove newline from subject */
        email.subject[strcspn(email.subject, "\n")] = '\0';

        insert_email(&heap, email);
    }

    else if (strcmp(command, "NEXT") == 0) {
        next_email(&heap);
    }

    else if (strcmp(command, "READ") == 0) {
        read_email(&heap);
    }

    else if (strcmp(command, "COUNT") == 0) {
        count_emails(&heap);
    }
}

return 0;
}

```

Google Gemini Generated Code — Baseline

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000

// Structure to represent an email
typedef struct {
    char category[30];

```

```

    char subject[100];
    char date[11]; // Format: MM-DD-YYYY
} Email;

// Structure for the list-based MaxHeap priority queue
typedef struct {
    Email emails[MAX_EMAILS];
    int size;
} MaxHeap;

// Function to assign priority ranks (lower number = higher priority)
int get_priority(const char* category) {
    if (strcmp(category, "Boss") == 0) return 1;
    if (strcmp(category, "Subordinate") == 0) return 2;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 4;
    return 5; // OtherPerson
}

// Function to convert MM-DD-YYYY string to a comparable integer value (YYYYMMDD)
int date_to_int(const char* date_str) {
    int month, day, year;
    sscanf(date_str, "%d-%d-%d", &month, &day, &year);
    return year * 10000 + month * 100 + day;
}

// Comparison function: returns 1 if email a has higher priority than email b
int compare_emails(Email a, Email b) {
    int p_a = get_priority(a.category);
    int p_b = get_priority(b.category);

    // Higher priority category comes first (lower rank number)
    if (p_a != p_b) {
        return p_a < p_b;
    }

    // If categories match, newer dates take precedence
    int date_a = date_to_int(a.date);
    int date_b = date_to_int(b.date);
    return date_a > date_b;
}

// Swap two emails in the heap
void swap(Email* a, Email* b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

```

```

// Sift-up operation for MaxHeap insertion
void sift_up(MaxHeap* heap, int idx) {
    while (idx > 0) {
        int parent = (idx - 1) / 2;
        if (compare_emails(heap->emails[idx], heap->emails[parent])) {
            swap(&heap->emails[idx], &heap->emails[parent]);
            idx = parent;
        } else {
            break;
        }
    }
}

// Sift-down operation for MaxHeap removal
void sift_down(MaxHeap* heap, int idx) {
    int last = heap->size - 1;
    while (2 * idx + 1 <= last) {
        int left = 2 * idx + 1;
        int right = 2 * idx + 2;
        int target = left;

        if (right <= last && compare_emails(heap->emails[right], heap->emails[left]))
        {
            target = right;
        }

        if (compare_emails(heap->emails[target], heap->emails[idx])) {
            swap(&heap->emails[target], &heap->emails[idx]);
            idx = target;
        } else {
            break;
        }
    }
}

// Insert a new email into the MaxHeap
void heap_insert(MaxHeap* heap, Email e) {
    if (heap->size >= MAX_EMAILS) return;
    heap->emails[heap->size] = e;
    sift_up(heap, heap->size);
    heap->size++;
}

// Remove the highest priority email from the MaxHeap
void heap_pop(MaxHeap* heap) {
    if (heap->size <= 0) return;
    heap->emails[0] = heap->emails[heap->size - 1];

```

```

    heap->size--;
    sift_down(heap, 0);
}

// Helper function to trim leading and trailing spaces from input strings
void trim_whitespace(char* str) {
    char* end;
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return;
    end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t' || *end == '\n' || *end == '\r'))
    {
        *end = 0;
        end--;
    }
}

int main() {
    MaxHeap inbox;
    inbox.size = 0;

    char line[256];
    while (fgets(line, sizeof(line), stdin)) {
        // Remove trailing newline
        line[strlen(line)] = 0;
        if (strlen(line) == 0) continue;

        if (strncmp(line, "EMAIL", 5) == 0) {
            char* token = line + 5;
            char* cat = strtok(token, ",");
            char* subj = strtok(NULL, ",");
            char* date = strtok(NULL, ",");

            if (cat && subj && date) {
                Email e;
                trim_whitespace(cat);
                trim_whitespace(subj);
                trim_whitespace(date);

                strcpy(e.category, cat);
                strcpy(e.subject, subj);
                strcpy(e.date, date);

                heap_insert(&inbox, e);
            }
        }
        else if (strcmp(line, "NEXT") == 0) {
            if (inbox.size > 0) {

```

```

        Email top = inbox.emails[0];
        printf("Next email:\n");
        printf("Sender: %s\n", top.category);
        printf("Subject: %s\n", top.subject);
        printf("Date: %s\n", top.date);
    }
}
else if (strcmp(line, "READ") == 0) {
    if (inbox.size > 0) {
        heap_pop(&inbox);
    }
}
else if (strcmp(line, "COUNT") == 0) {
    printf("There are %d emails to read.\n", inbox.size);
}
}

return 0;
}

```

Code Analysis

Correctness

Both programs understood the main idea of the assignment. They both created a heap using a vector, compared the sender categories, and used the date to decide which email should come first when the categories were the same.

One issue I noticed in both programs was the spelling of the category Important Person. The assignment uses *ImportantPerson* without a space, so I needed to make sure the final program matched the exact input the grader would use.

The ChatGPT code also gave me the basic heap operations I needed: inserting an email, looking at the highest-priority email, removing it, checking the size, and checking if the heap was empty.

The Gemini code had the main heap operations too, but it did not include the full command-processing portion of the program. Because the assignment requires the program to process EMAIL, NEXT, READ, and COUNT, I would have had to add more code before it could be used as the complete assignment.

Execution Time

Both programs use a standard heap structure, so their main heap operations have the same basic time complexity.

Inserting an email takes $O(\log N)$ because the new email may need to move up the heap.

Removing the highest-priority email takes $O(\log N)$ because the replacement email may need to move down the heap.

Looking at the highest-priority email takes $O(1)$ because it is stored at the front of the heap.

Counting the emails takes $O(1)$ because the vector keeps track of its size.

The two programs are therefore very similar when it comes to execution time. I did not see a major performance difference that would make one clearly better than the other.

Space Complexity

Both programs use a vector to store the emails in the heap. This means that if there are N emails, the heap uses $O(N)$ space.

Neither program creates another large data structure to hold duplicate copies of all the emails, so they are both reasonable in terms of memory usage.

Maintainability

For maintainability, I preferred the ChatGPT version. I found it easier to follow because the heap operations were separated into smaller helper functions such as `siftUp()` and `siftDown()`.

The Gemini version also works with the basic heap idea, but some of its naming was harder to follow. For example, `EmailEmailNode` and `PriorityQueueHeap` were more complicated names than I felt were necessary.

I also liked that the ChatGPT version called the heap simply `MaxHeap`, which matches the terminology used in the assignment.

Selection and Justification

I selected the ChatGPT program as my baseline.

The main reason I chose it was that I thought it gave me a better starting point for the assignment. The code was separated into smaller functions, which made it easier for me to understand what each part of the heap was doing. I could work on the heap itself separately from the input and command processing.

I also felt that the ChatGPT version would be easier for me to modify and debug. Since I was going to have to make changes to the generated code anyway, having a simpler structure made more sense for me.

Code Improvements Made

After choosing the ChatGPT program, I made several changes to turn it into my final program.

Corrected the Email Category Priority

Corrected the Email Category Priority

I changed the category names so that they match the assignment exactly. In particular, the assignment uses ImportantPerson, not Important Person.

I also changed the priority values so that the highest-priority category has the highest number. This makes the comparison work more naturally with a Maxheap.

The final priority order is:

Boss → Subordinate → Peer → ImportantPerson → OtherPerson

Added the Complete Command Processing

The original ChatGPT code only provided the heap itself. I added the main input loop so the program can read the test file one line at a time.

The final program can process:

EMAIL

NEXT

READ

COUNT

I also added checks for an empty heap so that NEXT and READ do not cause errors when there are no emails.

Added Email Parsing

I added code to separate the sender category, subject, and date from an EMAIL command.

I used the first comma to separate the category from the subject and the last comma to separate the subject from the date. This allows the subject to contain spaces, as required by the assignment.

I also added whitespace trimming so that extra spaces around the input fields do not become part of the stored email information.

Improved Comments and Documentation

I added a full prologue at the beginning of the program with my name, KUID, program description, inputs, outputs, creation date, revision date, collaborators, and sources.

I also added comments throughout the program explaining what the different sections and lines are doing. Since this assignment's rubric specifically asks for detailed comments, I wanted the comments to make it clear how the heap and command processing work.

I also identified the parts that were originally generated by ChatGPT and the changes that I made myself.

Result

Overall, I used the ChatGPT program as a starting point rather than submitting the generated code exactly as it was. I reviewed both programs, compared their correctness, execution time, space complexity, and maintainability, and then selected the one that gave me the structure I felt I could work with more easily.

The final version includes the MaxHeap, email priority system, date comparison, input parsing, and all of the required commands. I also tested and debugged the program so that it follows the required input and output format.